Agentic Generation of Process Models from Regulatory Texts

Catherine Sai[0009−0006−5971−8860] and Stefanie Rinderle-Ma[0000−0001−5656−6108]

Technical University of Munich, TUM School of Computation, Information and Technology, Garching, Germany, {catherine.sai, stefanie.rinderle-ma}@tum.de

Abstract. Process model generation is still mostly a manual task. A vast amount of process-relevant information is captured in textual sources such as regulatory documents and process descriptions. Hence, (semi-) automatic extraction of process model information from textual sources and translation into process models based on graphical notations such as BPMN are ongoing and, with the advent of generative AI, increasingly performant. However, existing approaches for process model generation from text are often limited to control flow aspects and require precise process descriptions as input. Regulatory documents describing processes pose different challenges than process descriptions and are widespread, highly relevant and of increasing volume in organizations. In this work, we exploit a generative AI architecture to present an approach that can automatically generate BPMN 2.0 process models from regulatory texts such as the GDPR. The approach is evaluated on five use cases with regulatory text and corresponding process models from different domains and with different challenges. This way, we can demonstrate that the proposed approach can automatically generate reference models from regulatory requirement documentations. When compared to existing approaches, the proposed approach results in an improvement of both, syntactic and semantic process model quality.

Keywords: Process Model Generation · Regulatory Process Description · Natural Language Processing · BPMN · LLM · Multi-Agent

1 Introduction

Visual process models are widely recognized for their role in helping employees understand, correctly execute, and improve business processes [7,18]. Their value is particularly evident when navigating changes, such as onboarding new staff or adapting processes to new requirements. The absence of clear process visibility can lead to significant operational inefficiencies and high costs, as noted by industry analysts who state, "The lack of visibility and understanding of company-wide and locally defined processes [...] can generate high costs [...]" [1]. Despite this

[1] https://blogs.sap.com/2022/08/10/5-reasons-why-process-modeling-is-the-operational-backbone-of-your-journey-to-process-analytics-initiatives/

proven need, many business processes still lack a formal visual representation. The primary bottleneck is the manual creation process, which is time-consuming and requires significant domain knowledge and specialized modeling skills. This "lack of qualified modelers" [24] is compounded by the fact that manual modeling is susceptible to errors, with industry model collections exhibiting "high error rates (between 10% and 20%)" [17]. Consequently, researchers have increasingly turned to automating the generation of process models from existing business resources, most notably textual descriptions [1, 13].

By contrast, this paper focuses on a particularly challenging, yet crucial source for process model generation, i.e., regulatory documents [28]. These documents, such as company regulations, legal statutes, or compliance rules, dictate what must be done, by whom, and under what conditions. However, they are typically written in natural language that is inherently ambiguous, complex, and not directly executable by IT systems [12]. In particular, a significant gap exists between these policy documents and the formal, structured process models required for effective business operations. Manually bridging this gap is a slow, expensive, and highly error-prone task that demands dual expertise in both the legal domain and process modeling. The challenges are amplified by the specific characteristics of legal texts, which often feature long, complex sentence structures, a high proportion of words irrelevant to the process flow, passive voice with implicit actors, and numerous cross-references to other paragraphs or documents [26]. Nevertheless, the utility of process modeling in this domain is clear; for instance, studies have shown that legal stakeholders can effectively understand BPMN models of legal processes, highlighting both the need and the feasibility of representing legal knowledge as formal processes [7].

Existing approaches to automated process discovery from text, whether based on traditional NLP or modern Large Language Models (LLMs), often fall short when confronted with the unique complexities of regulatory documents. Traditional rule-based methods may lack the flexibility to interpret ambiguous legal language, while LLM-based approaches (e.g., simple one-shot queries) can struggle to produce structurally complete and precise BPMN models. In particular, no existing work fully supports the automatic transformation of regulatory texts into BPMN 2.0 models with actor-responsibilities modeled as pools/lanes. This is a critical gap, particularly as the representation of roles and responsibilities is essential in regulatory contexts, especially for cooperative aspects in processes.

To address these limitations, this paper introduces a novel approach for generating comprehensive BPMN models from complex regulatory texts. The core contributions of this work are as follows:

- A **multi-agent** reasoning and refinement **architecture** that reflects on its own performance and iteratively improves its own results - completely automated.
- An **extended applicability** specifically tailored to handle the complexities of **regulatory documents**, capable of generating rich reference models that include key BPMN elements such as actor roles within pools.

- A **new benchmark dataset** consisting of five regulatory text-model pairs from diverse domains and complexity levels, released as an open-source resource to facilitate future research and comparative analysis in this area.
- An **automated evaluation framework** that provides objective and multi-faceted metrics for comparing generated process models, enabling a transparent assessment of an approach's capabilities and remaining challenges.

The remainder of this paper is structured as follows. Section 2 discusses related work in automated process model generation. The proposed approach, its novel components, and the analyzed aspects are presented in Sect. 3. This is followed by Sect. 5, which provides a comprehensive evaluation of our approach against existing work and on novel use cases. Finally, a discussion of the limitations and the conclusion are presented in Sect. 6 and Sect. 7, respectively.

2 Preliminaries and Related Work

This work is concerned with imperative process modeling, defining process control flows (and roles) explicitly, as exemplified by BPMN[2]. We adopt this focus because imperative models provide a visual representation of the process that is readily understood by diverse stakeholders, while the explicit assignment of roles and responsibilities is essential for effective organizational coordination. This combination of procedural clarity and defined responsibilities serves as a stable foundation for both, straightforward automation and seamless integration into operational workflows, which are the primary goals of this research. Therefore, we do not consider declarative approaches. To provide a structured overview, we differentiate prior work on information extraction and process model generation from text based on three key dimensions (cf. Tab. 1):

1. **Input Texts:** i) Process descriptions vs. ii) regulatory documents
2. **Method:** i) (partly) manual ii) rule-based NLP, iii) conversational LLM-based generation, iv) automated, single-turn interaction (one-shot, few-shot, agentic) LLM approaches
3. **Output Format:** i) (mapped) relevant text fragments, ii) (various) process models without roles/organizational structure (e.g. Mermaid) and iii) process model beyond control flow (e.g. BPMN with pools/lanes)

Early foundational efforts [10,26] show the feasibility of text-to-process modeling with classical NLP. The work by [26] is not fully automated and is based on formalization in addition to NLP techniques to create structured process models, aiming at the creation of reference models from regulatory texts. In contrast, the framework by Friedrich et al. [10] is automated and systematically addresses several key NLP challenges. However, despite its significance, this approach presents two limitations. Firstly, the method is based on fixed rules and not designed to handle regulatory documents. Secondly, from a technical perspective, its implementation poses limitations regarding reuse, adaptation, and

[2] Business Process Modeling and Notation, `bpmn.org`

Table 1. Comparison of Related Work on Information Extraction and Process Generation from Text

1. Input Text	2. Method	3. Output Format	Reference
i) Process desc.	ii) (rule-based) NLP	iii) process model with pools	[9, 10]
i) Process desc.	iii) conversational LLM	ii) process model w/o pools	[13, 14]
i) Process desc.	iv) automated LLM	i) (mapped) rel. text fragments	[5, 11, 21, 22]
i) Process desc.	iv) automated LLM	ii) process model w/o pools	[15]
ii) Reg. doc.	i) (partly) manual	ii) process model w/o pools	[8, 26]
ii) Reg. doc.	i) (partly) manual	iii) process model with pools	[3]
ii) Reg. doc.	ii) (rule-based) NLP	i) (mapped) rel. text fragments	[16,19,20,25,28]
ii) Reg. doc.	iv) automated LLM	iii) process model with pools	our approach

extension. These limitations in both domain applicability and technical accessibility highlight the need for a more modern and flexible approach incorporating current state-of-the-art reasoning solutions like LLMs.

Dimyadi et al. [8] use BPMN as part of an auditing system and Audrito and Ferraris [3] explore BPMN for legal education. The goal of both works aligns with ours (modeling regulatory text as BPMN), though their methods are human-driven; this shows the value of BPMN for legal compliance and the need for automation in the creation (which our LLM approach seeks to provide). Further regulatory compliance research such as [16, 19, 20, 25, 28] utilizes traditional NLP techniques to demonstrate use-cases of turning regulatory texts into process logic. They share the task of identifying and extracting relevant text fragments and either map these to process elements [19, 28] or other textual fragments [20, 25], or store the extracted requirements in knowledge graphs [16]. Their research highlights the challenges of regulatory texts and the importance of structuring regulatory text for automated compliance.

Leaving the regulatory application area, Bellan et al. [5] investigate GPT-3 as a large pre-trained LLM to automate the extraction of process elements from textual process descriptions. Neuberger et al. [21, 22] extract process information from text through trained (machine learning) models, targeting enriched process model construction, but without generating executable models. In [21], the authors aim at process information extraction through LLMs, with focus on a guide for structured prompting, analysis which aspects of a prompt improve the results and which do not. We incorporate their findings in our prompt design as detailed in Sect. 3. Using GPT-4 with one initial prompt, [11] examines its capabilities to generate BPMN models from textual process descriptions. However, the output of their LLM approach is not a BPMN model, but BPMN element pairs, e.g., `XOR (Proposal accepted)` $\Longrightarrow$ `Task1`. Furthermore, the evaluation is based on a manual mapping "between the entities identified in the dataset and the ones produced by GPT4" and restricted to identify the pair of their defined gold standard in the LLM output as well as actor activity pairs.

Our previous work investigates conversational process modeling to support iterative process model design and redesign with LLMs: [13] analyzes the ability of LLMs to create BPMN process models from process descriptions. In [14], we propose a method for process model redesign with iterative user feedback. This

work differs from previous work regarding input (regulatory documents instead of process descriptions) and output (pools and lanes in addition to control flow). Moreover, this work furthers the KPIs for assessing the output of created process models proposed in [13] into a novel evaluation metric.

The LLM-based approach ProMoAI [15] aims at generating BPMN models from text, but currently lacks full BPMN feature support for, e.g., pools and lanes, simplify gateways, are not designed for regulatory documents and lack functionalities like document upload. Despite these drawbacks, it is the most comparable solution to our approach, thus we will use [15] as a base model in the evaluation to compare the results of the approach proposed in this work. The emergence of generative AI has led to various (non-scientific) tools that aim to convert natural language directly into BPMN 2.0 models, seemingly offering functionality comparable to ProMoAi [15]. Examples include BPMN Sketch Miner[3] and BPMN GPT[4]. However, an initial evaluation reveals significant limitations when these tools are applied to regulatory documents, often resulting in invalid BPMN models.

3 CARB: Collaborative Agents for Regulatory BPMN

Figure 1 illustrates the **C**ollaborative **A**gents for **R**egulatory **B**PMN (CARB) generation approach proposed in this work. Input to CARB is a regulatory text that contains process-related information (cf. Fig. 1, I.). All instructions and queries to the LLM agents are generic, i.e., use case independent. The length of the input text is variable, it can span from a paragraph to a multi-page section of a regulatory document (cf. Tab. 2 for token size and other statistics of our tested use cases). The processing framework (cf. Fig. 1, II.) comprises a supervisor agent, coordinating four specialized worker agents, and a set of foundational tools for file system interaction and BPMN validation. The agents are set-up with the Gemini 2.5 Pro LLM, but this could be changed to another model in the future without further adjustments to the approach. Gemini 2.5 Pro is chosen because it has a big context window necessary for BPMN creation and shows strong reasoning and coding capabilities [5], consistently ranking high on benchmarks such as GPQA Diamond [6].

Architecture: The architecture of our approach is based on a hierarchical multi-agent system [7] designed to automate the generation and iterative refinement of process models from regulatory text in Business Process Model and Notation (BPMN) 2.0 format. For this a supervisor Large Language Model (LLM) agent acts as central orchestrator who manages a team of specialized LLM agents. Each agent is an LLM instance (Gemini 2.5 Pro) given a specific system prompt that

[3] https://www.bpmn-sketch-miner.ai/

[4] https://www.yeschat.ai/gpts-2OToEiyNiE-BPMN-GPT

[5] https://deepmind.google/models/gemini/pro/

[6] https://www.datacamp.com/de/blog/gemini-2-5-pro

[7] https://langchain-ai.github.io/langgraph/tutorials/multi_agent/agent_supervisor/

defines its role, constraints, and objectives. Through this separation of concerns, the complex task of BPMN creation from a regulatory text, is separated into a series of smaller, manageable problems. At the same time this architecture offers modularity which is advantageous for debugging, adjustments/extensions and is more robust, as e.g. the failure of one agent to perform his task does not lead to the failure of the whole approach.

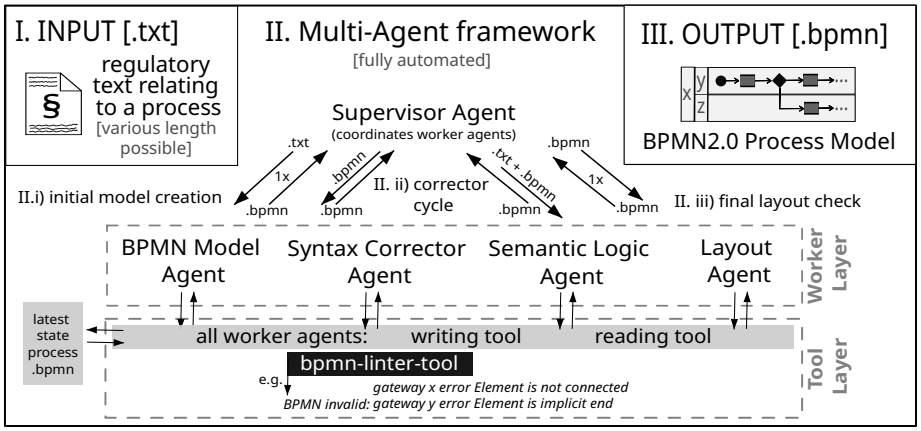

Fig. 1. CARB: Collaborative Agents for Regulatory BPMN generation

Tool Layer: All agents have access to the read- and write-file-tools, which offer standard functions for operations on text and BPMN/XML files. The Syntax Corrector Agent additionally uses the bpmn-linter-tool, which checks the BPMN file against the formal BPMN 2.0 specification. In order to automatically use the bpmnlint program as a tool, we wrap the bpmnlint command-line interface using a Python script.

Worker Layer: The **BPMN Modeler Agent** receives the regulatory text from the supervisor and creates the initial BPMN 2.0 XML structure. Its prompt mandates the creation of all necessary process elements (pools, tasks, gateways, events) and, critically, the corresponding visual layout information with *bpmndi*. It is designed to run only as initial step of the framework to deliver a *.bpmn* version that can be iteratively improved by the other worker agents. After the initial creation of the BPMN 2.0 model from the regulatory text, the **Syntax Corrector Agent** uses the bpmn-linter-tool to inspect the model. The tool delivers feedback on the structure and components of the model, assessing its syntactic validity. In case of an invalid result from the tool, the Syntax Corrector Agent is supplied with error insights, e.g. *Start Event x having an incoming sequence flow*. The Syntax Corrector Agent then takes these insights as input for its own reasoning, utilizing the reading and writing tool to correct the current process model XML file concerning syntactic validity. After correcting the XML, it re-runs the bpmn-linter-tool validation in a self-correction loop, if errors are reported (both new or persisting), it re-enters the self-correction loop

until the BPMN is syntactically valid. Following the syntactic validation check, the **Semantic Logic Agent** proceeds to analyze the model's logic against the source text. It compares the BPMN file against the original text, specifically verifying the implementation of conditional logic (gateways), obligations (must), permissions (may), and other business rules. If discrepancies are found, the Semantic Logic Agent corrects these, utilizing the reading and writing tools to modify the process model XML file. Finally, the **Layout Agent** is designed to inspect the visual representation of the syntactically and semantically improved BPMN process model. The aim of this agents is i.e. to eliminate element overlaps (especially labels), enforcing orthogonal (horizontal/vertical) sequence flows or ensuring elements are contained within their parent pools. In order to improve these elements, the agent utilizes the reading and writing tools to modify the coordinates within the *bpmndi* element of the XML file.

Supervisor: The Supervisor Agent coordinates the workflow with a combination of predefined sequence and execution-dependent routing. This control flow ensures that corrections are made in a logical order, preventing cascading errors. The process begins with the Supervisor invoking the BPMN Modeler Agent to produce the initial draft of the model. The supervisor is setup to only call the modeler once, which is strongly enforced in the execution script as otherwise the BPMN generation would start from an initial state and all previous correction effort would be lost. Following the initial BPMN creation, the supervisor initiates an iterative refinement cycle designed to achieve both syntactic and semantic correctness. First, the Supervisor calls the Syntax Corrector Agent, and after the syntax is validated, it calls the Semantic Logic Agent. If the Semantic agent makes a change to the file, the Supervisor restarts the loop by calling the Syntax Corrector Agent again. This ensures that any semantic modification has not inadvertently violated BPMN syntax rules. Similarly, if the Syntax agent identifies that the semantic corrected file is invalid and adjusts it, the resulting BPMN is sent again to the Semantic Logic Agent for reevaluation, ensuring that the conditions described in the text are still represented correctly in the XML file. This correction loop repeats until both Agents assess validity without corrections. Once the model passes through the primary correction loop without any further modifications (i.e., it is both syntactically and semantically sound), the Supervisor calls the Layout Corrector Agent for a single pass to clean up the visual diagram. As a final safety measure, the Supervisor invokes the Syntax Corrector Agent one last time to confirm that the layout adjustments did not corrupt the XML structure. The process terminates successfully when the final syntax check passes. To prevent infinite loops (e.g., if an agent repeatedly fails to fix an error), the Supervisor maintains an attempt counter for each agent and will halt the process if a predefined limit is reached, preserving the last working version of the model.

The final output (cf. Fig. 1, III.) is a syntactically and semantically improved BPMN 2.0 process model with specific emphasis on (regulatory) conditional modeling and actor responsibility inclusion.

4 Evaluation Metric

LLMs offer new opportunities in the automated creation of process models [14] and thus significantly reduce manual effort and improve consistency. However, an ongoing challenge is the holistic and objective evaluation of their results. To close this gap we propose a holistic evaluation metric, incorporating existing evaluation techniques and explaining their limitations. In order to access the quality of an LLM generated model, we identified 4 core aspects:

A. **Syntactic Quality [17, 27]:** Is the generated model compliant with the BPMN 2.0 standard?
B. **Semantic Quality [13,28]:** Does the model accurately represent the meaning, logic, and normative constraints of the source legal text?
C. **Layout:** How good is the Layout of the resulting BPMN visualization (Adherance to BPMN 2.0 Guidelines[8])? Are there flow or even task overlaps? Is the flow from left to right etc.?
D. **Stability of Results [21]:** How consistent are the generated models across multiple runs, given the non-deterministic nature of LLMs?

Although cost is also a known challenge for LLM-based approaches [21], continuous advances in model efficiency are rapidly lowering this barrier. Therefore, this work focuses on the three mentioned quality dimensions.

A. Syntactic Quality A generated process model must first be syntactically valid, adhering to the BPMN 2.0 modeling standards. For this validation, we leverage the Free Online BPMN Quality Analysis Service [9]. For this assessment, the generated .bpmn models are uploaded to the online service, which offers a visualization of the process, identifies modeling mistakes and checks the BPMN 2.0 Notation compliance. Examples of errors that are assessed include missing incoming/outgoing flows, gateways that are neither splits nor joins, or warnings like the information that a task label doesn't start with a verb. We deselect the consideration of multiple end events as warnings, but aside from that, keep all the checks they offer in the demo. The metrics we use from the BPMN Quality Analysis Service are the number of errors and warnings.

B. Semantic Quality Ensuring a model is syntactically correct is necessary, but insufficient. The ultimate measure of value is semantic correctness, i.e., how well a model represents the meaning of the source text. The core challenge in evaluating semantic correctness for process models generated from text, stems from a dual ambiguity. On the one hand, the inherent ambiguity of natural language means that the same (regulatory) text can be interpreted differently by various audiences. On the other hand, the representational freedom in process modeling allows for the same process being modeled in different ways. For regulatory

[8] `https://www.bpmnquickguide.com/view-bpmn-quick-guide/`
[9] `https://freebpmnquality.github.io/`

documents these ambiguity challenges are even greater due to the interpretable legal language being used and the complexity of the contained topics.

Our approach builds upon foundational research in automated BPMN generation, compliance checking and LLM evaluation. [10], used *Graph Edit Distance (GED)* for measuring process model similarity. GED calculates the minimum-cost transformation of one graph into another, considering node/edge insertions, deletions, and substitutions. Friedrich's enhanced this by incorporating the semantic similarity of element labels for the matching. There are two main reasons why GED is not suitable for our approach. Firstly, it heavily depends on a gold standard model and views this as the only truth. Hence, GED is incapable to verify that a model is a faithful representation of a regulatory text, calculating the models similarity, but not the link between the model element and the specific legal clause that justifies it. Secondly, even if a gold standard model exists, GED cannot validate if an activity is obligatory or permitted (e.g., with GED two element pairs can seem a perfect match, even if the text required one to be mandatory and the other optional). While our framework also uses a gold standard for the comparison of BPMN element existence (e.g., actors, activities, event. etc.) and placement (e.g., activity placed in correct actor pool), we additionally include the regulatory source text for the semantic evaluation as the ultimate ground truth fro complex conditional requirements. Our previous work in [13] provides KPIs for task extraction from text. In [28], we provide a fitness score to quantify the relevance of text to a model and a cost score to penalize three specific violations, namely (1) missing obligatory activities, (2) wrong order of activities and (3) wrong resource allocation of activities. We adapt and extend this prior metrics, introducing the Regulatory Semantics Score (RSS), which is comprised of three components, where we explicitly distinguish between the *completeness* covering the existence of an element, its *correctness*, i.e., position/relations, and *traceability* measuring the percentage of generated elements that are present in the gold standard in order to asses the grounding of elements in the source text to prevent hallucination and over-compliance (cf. [25]). This differentiation transparently identifies whether an error in the generated model is caused by missing information retrieval from the text (completeness, i.e., element not existent in model) or due to its processing in relation to the other elements (correctness) or created by the LLM as, e.g., a cause of too much creativity (traceability).

Completeness checks if all elements that should exist actually exist in the generated model. This can be done via the *recall* for the following elements:

(i) % of **actors [AR]** that correctly exist in the model (e.g. as BPMN Pools or Lanes)
(ii) % of **activities [AY]** that correctly exist in the model (cf. [13], only obligatory activities: [28])
(iii) % of **events [EV]** that correctly exist in the model (e.g. start, end, intermediate events)
(iv) % of **AND gateways [AG]** that correctly exist in the model

(v) % of **XOR gateways [XG]** that correctly exist in the model
(vi) % of **data objects [DO]** that correctly exist in the model

The completeness check is automated through a semantic similarity score with a threshold to define if, e.g., a role name in the generated model is semantically similar to the desired label in the gold standard model or not. Using Sentence Transformer [23] embeddings and cosine similarity, for each element-type (i, ii, etc.) in the gold standard, the best-matching element (with a score above the element-types similarity threshold) in the generated model is identified. This label similarity check is performed for elements (i)-(iii) and (vi). For the gateways we use a count at this point, as gateways (with the exception of XOR-splits) are not required to have a label and thus cannot be distinguished from one another. Thus, here we only verify the number of AND and XOR gateways that exist in the model. Their correct placement is checked in the next step.

Correctness checks the following aspects in the resulting process model:

(i) % of activities with correct **actor assignment [AA]**
(ii) % **control flow [CF]** (are activities, gateways, events and objects connected in the correct sequence?)
(iii) % of correct **decision logic [DL]** elements (meaning the conditions on the outgoing paths of XOR split)
(iv) % **conditional logic [CL]** (specification correctly modeled?, here we pay special attention to regulatory specifications like obligations)
  (a) % of **temporal** constraints correctly translated into gateway logic and event triggers
  (b) % of **obligatory** actions (e.g. shall, should, must, has to [4]) modeled as part of a non-optional execution path
  (c) % of **prohibited** actions (e.g. must not) correctly modeled as impossible paths or explicit exception flows
  (d) % of **permitted** actions (e.g. can, may) correctly modeled as optional paths (e.g. following a choice gateway)

For the correctness evaluation, we compare the generated model against both, the gold-standard reference model and the original regulatory text. The methodology involves two primary stages: structural comparison of relational pairs (i-iii) and logical validation (iv). For i), we compare actor-activity-pairs extracted from the gold standard to semantically similar actor-activity-pairs in the generated model. Both elements of the pair have to be semantically similar to their counterpart in a comparison pair in order to be considered correct. We again use sentence-embeddings and cosine similarity (with static element thresholds) to assess the semantic similarity between element labels. Similarly for ii) we extract the labels of all direct connections between elements (tasks, events, gateways) as (source, target) pairs. The pairs are checked the same way as described for the i) pairs, based on semantic similarity. This evaluation relies on the elements being named. If e.g., a gateway in the model does not have a label, it cannot be included as a source or target in a control flow pair. For iii), the approach checks

if XOR-gateway-split labels are semantically similar and if that is the case if the outgoing sequence flows are also semantically similar. So for this decision logic check, the pair that is compared between gold standard and generated model consists of the gateway label and the sequence flows that exit the gateway. Note that especially (i)-(iii) depend on the existence of the elements. If e.g., an actor was not identified at all this will not only affect the completeness score, but also the correctness score as there can be no actor-activity assignment for the missing actor. Aspect (iv), is evaluated differently. Here, we leverage an LLM for its reasoning capabilities in assessing the conditional logic modeling. The LLM is designed to identify business rules and constraints from the source text and analyze them against the generated model to verify if those rules are implemented logically and correctly.

Traceability checks the following numbers:

(i) % of **activities [AI]** that correctly exist in the model vs. all activities that exist in the model
(ii) % of **actors [AP]** that correctly exist in the model vs. all actors that exist in the model

This dimension ensures auditability by verifying that model elements are explicitly grounded in the source text, preventing models to rank high that contain unnecessary elements (e.g. hallucinations by the LLM). The most suitable metric is *precision* as it measures the percentage of correct elements (that should exists in the model) over all the elements generated. To keep the evaluation concise we limit this check to the actor and activity elements, as they usually contain the most essential information of a process.

Calculating an overall Regulatory Semantics Score The overall RSS score is a weighted average of the scores from each dimension:

$$\mathbf{RSS} = \text{Score}_{\text{Completeness}} \cdot w_c + \text{Score}_{\text{Correctness}} \cdot w_r + \text{Score}_{\text{Traceability}} \cdot w_t \quad \text{where} \quad \sum w_i = 1 \quad (1)$$

The weights (w_i) can be adjusted based on the evaluation's focus. We recommend a focus on correctness, as this contains the most advanced aspects (e.g., conditional logic). This score provides a quantitative measure of similarity between the legal text semantics and the generated model.

C. Layout To assess the Layout of the resulting BPMN visualization, we use the generated BPMN as input for a LLM (Gemini 2.5 Pro), designed to identify and count defined criteria of the BPMN 2.0 Layout Guidelines[10] (e.g. flow, label or element overlaps, non-orthogonal flows, etc.).

[10] `https://www.bpmnquickguide.com/view-bpmn-quick-guide/`

D. Stability of Results LLMs are known for their non-deterministic output, which raises concerns about the stability of results. To address this, we adopt a methodology analogous to [21]. We run the model generation process multiple times (we chose three iterations) for the same input setup (text and prompt). By calculating the standard deviation of the semantic evaluation results, we can quantify the variance of the generated models. A low standard deviation indicates the stability of the generation process and its results.

The proposed multi-criteria evaluation systematically assesses syntactic correctness, semantic fidelity (via the RSS score), layout of the resulting visualization, and result stability. It aims at providing a holistic, quantitative understanding of the utility of a process model created from legal text. The evaluation metric integrates established methods with novel, text-grounded semantic checks, paving the way for the trustworthy application of LLMs in the legal domain.

5 Evaluation

The CARB approach is implemented in Python using the LangChain[11] framework. It is publicly available at `https://github.com/CatherineSaiTUM/RegText2BPMN`, and can be run through a simple google colab setup. In the following, we evaluate CARB based on different data sets.

5.1 Data Sets, Gold Standard Creation, and Base Models

We use gold standard process models created from texts as the comparison between model and model is more objectifiable than from text to model [10]. The data set consists of five BPMN process models and their corresponding texts from five different regulatory documents of various domains. For two processes, the regulatory text and corresponding process model exist, but are adjusted by us for higher consistency with the BPMN 2.0 standard and better representation of the given information. The two adjusted models are taken from [2,28] (GDPR) and [6,28] (Smart Meter). To validate the necessity for adjustment of the models, we check the original models for syntax quality (cf Sec. 4) using the Free BPMN Quality tool. In both cases, errors are detected such as missing connection to sequence flow and modeling organizational aspects as task labels instead of using pools or lanes. The adapted process models are checked without any errors or warnings by the Free BPMN Quality tool. For the three other use cases, we create text-process model pairs without an existing process model from the regulatory text. In detail, we first identify text passages in regulatory documents (like laws, regulations, and enactments) that include the description of a process. The texts are then manually annotated into categories actor, action, condition/ process relevant information, and process irrelevant information. Parallel to this, the texts are provided to an LLM (Gemini 2.5 Pro Chat) to create an initial BPMN model. Subsequently, the initial BPMN model and the annotated information

[11] `https://www.langchain.com/`

are manually combined to improve the BPMN model and ensure its adequate representation of the all relevant information. Finally, the five resulting process models are quantitatively evaluated by the automated syntax quality metric as described in Sec. 4. Furthermore, the authors reviewed the semantical fit and layout of the resulting models as a qualitative assessment, since an automated analysis of these aspects is only possible when a gold standard model already exists, and in this case such models were being created.

An overview of the use cases with information on the textual and model data is displayed in Tab. 2. As mentioned in [9] the performance an approach depends on the number of sentences or average sentence length contained in the input text. Thus, we show statistics on the input complexity of each use case through the number of sentences n and the average length of sentences measured in number of tokens l. For the assessment of gold standard (GS) model complexity, we extend the measures provided in [9] by roles, resulting in 4 values: the size of the models in terms of nodes N (activities and events), gateways G, edges E, and roles R. Use case III. (Blood Donor Selection) is by far the longest input text as a whole section of a regulation concerned with the details of the corresponding process. the manually created process model has 39 nodes, 14 gateways, and 54 edges. Use case V. (Customer Due Diligence) is the shortest in terms of number of sentences, but the sentences are long with on average 34.5 tokens, which makes it very challenging to extract information.

Table 2. Input text and gold standard model characteristics for the 5 use cases

Use Cases			**Text Input**		**GS Model**			
name	domain	source model	n	l	N	G	E	R
I. Smart Meter	energy mgmt.	adj. from [6, 28]	14	25.9	28	9	36	4
II. GDPR	data privacy	adj. from [2, 28]	23	28.1	21	13	38	4
III. Blood Donor Selection	medical procedure	newly created	524	22.1	39	14	54	4
IV. Health Data Exchange	medical data	newly created	19	24.9	19	3	19	4
V. Customer Due Diligence	finance	newly created	12	34.5	24	5	28	3

We compare CARB to two existing solutions as base models, i.e., 1) ProMoAi [15] and 2) Gemini 2.5 Pro Chat with the prompt design from [21]. The LLM chosen for processing within ProMoAi is also Gemini 2.5 Pro for comparability reasons. For base model 1) *ProMoAi*, it should be noted that the resulting process models are focused on control flow, neglecting actors, gateway labels and events (except start and end event) and thus create more simplistic process models. For the base model 2), we incorporate the prompt designed by [21], using the sections that are identified as 'useful' in their prompt evaluation and adjust what is necessary for BPMN model creation (as their prompt aims at a textual output/ process information extraction). The adjusted prompt is used in the Gemini 2.5 Pro Chat resulting in .bpmn output.

5.2 A. Syntactic Quality

Concerning the syntactic evaluation results (cf. Tab. 3) CARB outperforms Gemini Chat. Compared to ProMoAi CARB has less warnings, but leads to some

errors. These errors marked for 4 out of the 15 runs with CARB (resulting in the overall average errors of 0.4), occur due to multiple start events within a pool, e.g., 2 errors for one run of the GDPR are caused by two intermediate message events wrongly being modeled as start message events. However, ProMoAI does not run into errors as their detected process logic is rather simplistic, i.e., there are no events and pools in the models.

Table 3. Syntactic Evaluation

Use Case	GeminiChat		ProMoAi		CARB	
	Avg. Errors	Avg. Warn.	Avg. Errors	Avg. Warn.	Avg. Errors	Avg. Warn.
I. Smart M.	14.00	0.33	0.00	1.67	0.00	0.33
II. GDPR	22.67	0.00	0.00	4.33	0.67	0.33
III. Blood D.	17.33	0.33	0.00	1.67	0.67	0.00
IV. Health D.	20.67	0.00	0.00	1.33	0.00	0.67
V. CDD	7.33	0.67	0.00	3.00	0.67	1.67
Overall	**16.40**	**0.27**	**0.00**	**2.40**	**0.40**	**0.60**

5.3 B. Semantic Quality

The %-values shown are the average for all 3 runs per use case for each of the evaluation aspects introduced in Sec. 4. The results per execution run as well as the absolute values and the automated evaluation script are included in the code repository. As correctness is the core of the semantic evaluation, it is weighted with 0.6 in the RSS score, while completeness is weighted with 0.3 and traceability with 0.1.

Table 4. Semantic Evaluation [average values for 3 executions]

	Use Case	completeness						correctness				trace.		RSS
		AR	AY	EV	DO	AG	XG	AA	CF	DL	CL	AP	AI	
Gemini	I. Smart Meter	0.83	0.24	**0.36**	N/A	0.33	0.33	0.20	**0.05**	0.07	0.59	0.92	**0.64**	**0.34**
	II. GDPR	**1.00**	0.50	0.27	N/A	0.00	0.47	0.50	0.07	0.33	0.60	**1.00**	0.63	0.44
	III. Blood D.	0.33	0.22	0.22	N/A	N/A	0.11	0.07	0.10	0.14	0.35	0.36	0.46	0.21
	IV. Health D.	**1.00**	0.75	0.27	0.00	N/A	0.00	0.63	0.21	0.00	0.48	0.78	**0.95**	0.41
	V. CDD	**0.78**	0.24	0.07	0.00	N/A	0.20	0.19	0.05	0.17	0.29	**0.78**	0.54	0.25
ProMoAi	I. Smart Meter	0.00	0.27	0.15	N/A	**0.67**	**0.57**	0.00	0.00	0.00	0.61	0.00	0.48	0.22
	II. GDPR	0.00	0.47	0.09	N/A	**1.00**	**1.00**	0.00	0.03	0.00	0.72	0.00	0.26	0.28
	III. Blood D.	0.00	0.27	0.00	N/A	N/A	**0.61**	0.00	0.06	0.00	0.49	0.00	0.28	0.16
	IV. Health D.	0.00	0.58	0.00	0.00	N/A	**1.00**	0.00	0.04	0.00	0.74	0.00	0.72	0.25
	V. CDD	0.00	0.36	0.00	0.00	N/A	**1.00**	0.00	0.00	0.00	0.62	0.00	0.26	0.19
CARB	I. Smart Meter	**0.92**	**0.33**	0.23	N/A	0.00	**0.57**	**0.29**	0.04	**0.20**	**0.86**	**1.00**	0.51	**0.34**
	II. GDPR	**1.00**	**0.63**	**0.55**	N/A	0.67	0.83	**0.63**	**0.14**	**0.48**	**0.71**	**1.00**	**0.65**	**0.56**
	III. Blood D.	**0.42**	**0.48**	**0.36**	N/A	N/A	0.33	**0.11**	**0.18**	**0.29**	**0.71**	**0.61**	**0.47**	**0.31**
	IV. Health D.	**1.00**	**0.79**	**0.52**	0.00	N/A	0.56	**0.75**	**0.48**	**0.33**	**0.80**	**0.82**	0.85	**0.58**
	V. CDD	0.67	**0.43**	**0.19**	0.00	N/A	0.93	**0.33**	**0.08**	**0.33**	**0.86**	0.56	**0.65**	**0.38**

For *completeness* scores, CARB performs clearly better than base models in generating necessary activities (AY) and events (EV). For actor (AR) identification it is slightly better than Gemini Chat (superior in 2 use cases, inferior in one), but here the results are comparable with the Gemini Chat approach. None of the models created data objects (DO). For gateways (AG) and (XG), CARB performs better than Gemini Chat, but ProMoAi seems superior to both. However, for the interpretation of these results it is important to note, that ProMoAi only includes activities, (unlabeled) gateways and start- and end event in their

process models, thus it is to be expected to generate many gateways as they do not include other modeling options. The completeness is only a simplistic existence check, counting the number of gateways per type existing in the model. The check concerning correct decision logic is evaluated in correctness.

The evaluation of *correctness* demonstrates that CARB achieves the highest performance scores across all four tested aspects and five use cases (with exception of one CF value). As all three compared approaches incorporate LLM-based reasoning, they each demonstrate satisfactory performance for the conditional logic (CL). Nevertheless, CARB's specific architecture ensures its superior performance in this category as well.

The *Traceability* metric measures how well generated actors and activities are grounded in the source text; low scores indicate potential hallucinations or an incorrect level of detail. While CARB again achieves the best results, its performance is more comparable to Gemini Chat on this metric, showing a smaller advantage than was observed for completeness and correctness.

The *RSS* offers an overall performance indicator per use case. It clearly supports the expectation that the blood donor use case III was the most difficult to model due to its extensive input text. All 3 approaches perform worst on this use case. Use case V (CDD) was the second most challenging, likely due to the length of its sentences.

5.4 C. Layout

Table 5. Layout Evaluation Scores

Use Case	Gemini	ProMoAi	CARB
I. Smart M.	0.70	0.20	0.50
II. GDPR	0.20	0.10	0.30
III. Blood D.	0.40	0.10	0.20
IV. Health D.	0.90	0.15	0.65
V. CDD	0.65	0.10	0.33
Avg. Score	**0.57**	**0.13**	**0.40**

The layout quality of the generated process models is evaluated based on their visual representation (.png). For each use case, only the first generated model is considered. An overall layout score, ranging from 0.0 (very poor) to 1.0 (perfect), is assigned by an LLM (Gemini 2.5 Pro) according to specified quality criteria (cf. Section 4). In terms of layout, CARB does not outperform the baseline methods. It scores higher than ProMoAi and lower than Gemini. One potential reason for CARB's lower performance is the increased complexity and element count in its generated models, which introduces more opportunities for layout errors. However, the layout agent itself yields only marginal improvements, suggesting an inherent difficulty in using a reasoning agent to interpret positional information from an XML source. Therefore, a deterministic, rule-based approach may be more suitable for this task.

5.5 D. Stability of Results

To assess the stability of the generated outputs, we execute the five use cases three times with each approach. Table 6 presents the resulting average standard deviation for the 12 semantic evaluation results. Overall, the models demonstrate high stability, evidenced by low standard deviations. The primary source

of instability is the existence of AND-Gateways (AG), which shows the highest standard deviation. This occurs because in most runs no gateways are generated, but in rare instances, the models includes two, creating the variance.

Table 6. Average Standard Deviation Across All Use Cases

Approach	**completeness**						**correctness**				**trace.**	
	AR	AY	EV	DO	AG	XG	AA	CF	DL	CL	AP	AI
Gemini	0.07	0.08	0.11	0.00	0.29	0.09	0.05	0.07	0.08	0.06	0.17	0.15
ProMoAi	0.00	0.09	0.00	0.00	0.29	0.08	0.00	0.03	0.00	0.20	0.00	0.08
CARB	0.12	0.10	0.10	0.00	0.29	0.16	0.05	0.07	0.22	0.09	0.11	0.15

6 Discussion

A primary limitation concerns the *applicability* of our approach to different types of input texts. CARB is specifically designed for regulatory documents that contain procedural logic, such as sequences, and conditional rules. For texts that are purely declarative and lack such procedural information, our method is less suitable. In these cases, alternative approaches like declarative process model extraction, as proposed by van der Aa et al. [1], would likely yield more appropriate results. Furthermore, while we expect that CARB's dynamic architecture could model non-regulatory texts by simply omitting unneeded components like conditional logic, this was not verified. The applicability of our approach to non-regulatory documents was outside the scope of our evaluation and remains an area for future work.

Another consideration is the *intended usage* of the results. The models generated by CARB should not be viewed as final, fully legally compliant representations of the text. As discussed in Section 4, the inherent ambiguity in natural language makes it unreasonable to expect a generated model to be a 100% perfect match to a human-created gold standard. Instead, they are designed to serve as initial reference models. Their primary purpose is to aid stakeholders with a foundational, procedural understanding of complex regulations and to act as an aid in the manual modeling of regulatory process logic. We strongly recommend that any generated model undergoes a thorough review by both a legal expert and a process modeling expert before it is considered a ground truth or integrated into business operations.

Finally, a challenge in our evaluation is the subjective nature of *process granularity*. The level of detail captured in a process model can vary between modelers, and our system may make different choices than a human expert. For instance, a single, high-level activity in the gold standard, such as 'Forward request to system', is represented by a generated model in a finer granularity, splitting the 'request' into two distinct activities 'Initiate Activation' and 'Initiate Deactivation'. This discrepancy is not an error, but a difference in the level of abstraction.

7 Conclusion

This work introduces CARB, a novel approach to automatically generate BPMN process models from regulatory texts. The novel contributions of this work are twofold. First, CARB is designed to interpret modalities, such as obligations and permissions, inherent in legal documents. Second, it utilizes a Supervisor Multi-Agent framework that decomposes the complex task of process model generation with layout information into specialized sub-tasks for individual agents. This architecture enables iterative quality checks and self-correction, significantly improving the final output. The evaluation demonstrates that CARB outperforms existing methods in extracting model information beyond control flow, e.g., pools and lanes, and at the same time offers comparable or even superior performance w.r.t. syntactic correctness and semantic quality. For future work, we plan to engage legal experts to further validate the semantic accuracy of both, the gold standard and the generated models. Furthermore, we will enhance the layout agent by developing a rule-based tool to address the persistent challenge of automated process model visualization without flaws.

References

1. van der Aa, H., Di Ciccio, C., Leopold, H., Reijers, H.A.: Extracting declarative process models from natural language. In: Advanced Information Systems Engineering. pp. 365–382 (2019)
2. Agostinelli, S., Maggi, F.M., Marrella, A., Sapio, F.: Achieving GDPR compliance of BPMN process models. In: CAiSE Forum. pp. 10–22 (2019)
3. Audrito, D., Ferraris, A.F.: Legal design through business process model and notation (BPMN): a digital services act case-study. In: Joint Ontology Workshops (2023)
4. Bellan, P., Dragoni, M., Ghidini, C.: Combining natural language processing approaches for rule extraction from legal documents. In: AI Approaches to the Complexity of Legal Systems Workshops. pp. 287–300 (2017)
5. Bellan, P., Dragoni, M., Ghidini, C.: Extracting business process entities and relations from text using pre-trained lms. In: Enterprise Distributed Object Computing Conference. pp. 182–199 (2022)
6. Böhmer, K., Stertz, F., Hildebrandt, T., Rinderle-Ma, S., Eibl, G., Ferner, C., Burkhart, S., Engel, D.: Application and testing of business processes in the energy domain. In: BTW. pp. 25–32 (2017)
7. Capuzzimati, F., Violato, A., Baldoni, M., Boella, G.: Business process management for legal domains: Supporting execution and management of preliminary injunctions. In: Legal Knowledge and Information Systems. pp. 149–152 (2015)
8. Dimyadi, J., Clifton, C., Spearpoint, M., Amor, R.: Computerising regulatory knowledge for building engineering design. Journal of Computing in Civil Engineering (2024)
9. Friedrich, F.: Automated generation of business process models from natural language input (2010)
10. Friedrich, F., Mendling, J., Puhlmann, F.: Process model generation from natural language text. In: Advanced Information Systems Engineering. pp. 482–496 (2011)

11. Grohs, M., Abb, L., Elsayed, N., Rehse, J.: Large language models can accomplish business process management tasks. In: Business Process Management Workshops. pp. 453–465 (2023)
12. Hashmi, M., Governatori, G., Lam, H., Wynn, M.T.: Are we done with business process compliance: state of the art and challenges ahead. Knowl. Inf. Syst. pp. 79–133 (2018)
13. Klievtsova, N., Benzin, J., Kampik, T., Mangler, J., Rinderle-Ma, S.: Conversational process modelling: State of the art, applications, and implications in practice. In: BPM Forum. pp. 319–336 (2023)
14. Klievtsova, N., Kampik, T., Mangler, J., Rinderle-Ma, S.: Conversationally actionable process model creation. In: Cooperative Information Syst. pp. 39–55 (2024)
15. Kourani, H., Berti, A., Schuster, D., van der Aalst, W.M.P.: Promoai: Process modeling with generative ai. In: IJCAI Demo Track. pp. 8708–8712 (2024)
16. Kruiper, R., Kumar, B., Watson, R., Sadeghineko, F., Gray, A.J.G., Konstas, I.: A platform-based natural language processing-driven strategy for digitalising regulatory compliance processes for the built environment. Adv. Eng. Informatics p. 102653 (2024)
17. Mendling, J.: Metrics for Process Models: Empirical Foundations of Verification, Error Prediction, and Guidelines for Correctness. Springer (2008)
18. Mendling, J., Reijers, H.A., van der Aalst, W.M.P.: Seven process modeling guidelines (7PMG). Inf. Softw. Technol. pp. 127–136 (2010)
19. Muram, F.U., Javed, M.A., Kanwal, S.: Facilitating compliance of process models with critical system standards using nlp. In: Evaluation of Novel Approaches to Software Engineering. pp. 306–313 (2021)
20. Nake, L., Kuehnel, S., Bauer, L., Sackmann, S.: Towards identifying gdpr-critical tasks in textual business process descriptions. In: GI Jahrestagung. pp. 1895–1908 (2023)
21. Neuberger, J., Ackermann, L., van der Aa, H., Jablonski, S.: A universal prompting strategy for extracting process model information. In: Conceptual Modeling. pp. 38–55 (2024)
22. Neuberger, J., Ackermann, L., Jablonski, S.: Beyond rule-based named entity recognition and relation extraction for process model generation from natural language text. In: Cooperative Information Systems. pp. 179–197 (2023)
23. Reimers, N., Gurevych, I.: Sentence-bert: Sentence embeddings using siamese bert-networks. In: Empirical Methods in Natural Language Processing, EMNLP (2019)
24. Rosemann, M.: Potential pitfalls of process modeling: part A. Bus. Process. Manag. J. pp. 249–254 (2006)
25. Sai, C., Winter, K., Rinderle-Ma, S.: Detecting deviations between external and internal regulatory requirements for improved process compliance assessment. In: Advanced Information Systems Engineering. pp. 401–416 (2023)
26. Wang, H.J., Zhao, J.L., Zhang, L.: Policy-driven process mapping (PDPM): discovering process models from business policies. Decis. Support Syst. **48**(1), 267–281 (2009)
27. Weske, M.: Business Process Management: Concepts, Languages, Architectures. Springer (2007)
28. Winter, K., van der Aa, H., Rinderle-Ma, S., Weidlich, M.: Assessing the compliance of business process models with regulatory documents. In: Conceptual Modeling. pp. 189–203 (2020)